

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 4

Software Design

Author: Maria Moruțan

Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

May 2026

Contents

1	Problem definition	2
1.1	Users and roles	2
2	Architecture	2
2.1	Client-Server Architecture and Service Boundaries	2
2.2	Real-Time STOMP Over WebSocket Communication Architecture	3
3	System diagrams	3
3.1	Use case diagram	3
3.2	Entity-relationship diagram	4
3.3	Class diagram	5
3.4	Sequence diagram	5
4	Functional requirements	6
4.1	Preservation of Core Functionalities (Assignments 1–3)	6
4.2	Client-Server Separation API Endpoint Interaction	6
4.3	Dynamic Internationalization (i18n)	6
4.4	Real-Time WebSocket Chat Messaging System	7
4.5	Distributed Graceful Error Handling and Consistent Payloads	7
5	Design patterns	8
5.1	Broker Pattern (Architectural)	8
5.2	Mediator Pattern	8
5.3	Observer Pattern (Event-Driven)	8
6	Implementation details	8
6.1	STOMP Over SockJS Frame Handshaking Loop	8
6.2	Global Request Interception and Exception Management	8
7	Unit testing	9

1 Problem definition

This document describes the design and implementation of Assignment 4, which extends the distributed Pet Management System to create a collaborative, multi-user real-time environment. The foundational business domain remains a Pet Shop enterprise managing a catalog of animals, inventory products, credential-secured identities, and format-flexible data exports.

The core objective of this assignment is to upgrade the system from isolated transactional workflows into a fully interactive client-server distributed application. This is achieved by implementing real-time bidirectional messaging channels, integrating dynamic system runtime internationalization (i18n), and deploying a consistent fault-tolerance ecosystem that maps server-side anomalies to user-friendly UI component views without interrupting the client process layout.

1.1 Users and roles

The system enforces role-based access control rules across three user classes, each interacting with the application layer differently:

- **Visitor** – An unauthenticated guest user. Visitors can view the public catalog indices for available pets and retail items but cannot modify data states or establish connection links to the real-time chat application.
- **Administrator** – An authenticated power-user who possesses full permission privileges to run create, read, update, and delete (CRUD) routines on all entities. Administrators can access all administrative panels, toggle system languages, and participate in the live chat module.
- **Seller** – An authenticated domain-specific operator. Sellers can execute data insertion routines to expand the pet and item catalog but are restricted from running record deletion or critical modification workflows. Sellers have full authorization to use the real-time communication system.

Session verification is maintained via signed JSON Web Tokens (JWT) generated by the **auth-service**. These token identities dictate structural template options rendered inside the client UI layout.

2 Architecture

2.1 Client-Server Architecture and Service Boundaries

The application uses a decoupled client-server architecture pattern where the presentation layer is isolated from the underlying data management microservices:

- **Client Layer (frontend-service)** – Running on port 8080, this service functions as an autonomous user interface manager and gateway. It serves localized HTML views, manages client session lifetimes, and runs async JavaScript components (**chat.js**, **items.js**). It connects to backend layers using exposed REST endpoints and stateful WebSocket tunnels.

- **Server Ecosystem (Backend Microservices)** – Composed of isolated, stateless functional applications running independently on dedicated ports:
 - **pet-service** (Port 8081) – Exposes APIs for pet catalog search, indexing, and item modifications against **pet_db**.
 - **item-service** (Port 8082) – Exposes inventory control APIs, resolving missing entity details across microservice boundaries on demand.
 - **auth-service** (Port 8083) – Manages registration, hashing, and token generation routines.
 - **notification-service** (Port 8084) – Asynchronously processes cross-origin event messaging.
 - **export-service** (Port 8085) – Transforms active catalog parameters into structured JSON, CSV, or XML documents.

2.2 Real-Time STOMP Over WebSocket Communication Architecture

Standard HTTP transactional cycles are augmented with a stateful bidirectional data network built on the STOMP architecture layer, using a SockJS fallback pipeline to support client connections across different environments.

This messaging configuration operates via a dedicated routing registry:

- The client application initializes a persistent TCP channel connection directed at the unified server handshake endpoint path located at **/ws-chat**.
- Inbound text frames containing client payloads prefixed with **/app** are immediately intercepted by specific method handlers inside the application context.
- Outbound delivery is routed through an internal simple message broker configured with the **/topic** prefix, which automatically broadcasts message frames to all active channel subscribers simultaneously without requiring a page refresh.

3 System diagrams

3.1 Use case diagram

The system use case layout includes user interaction capabilities for live chat rooms, language selection preferences, and secure management actions across different user access roles:

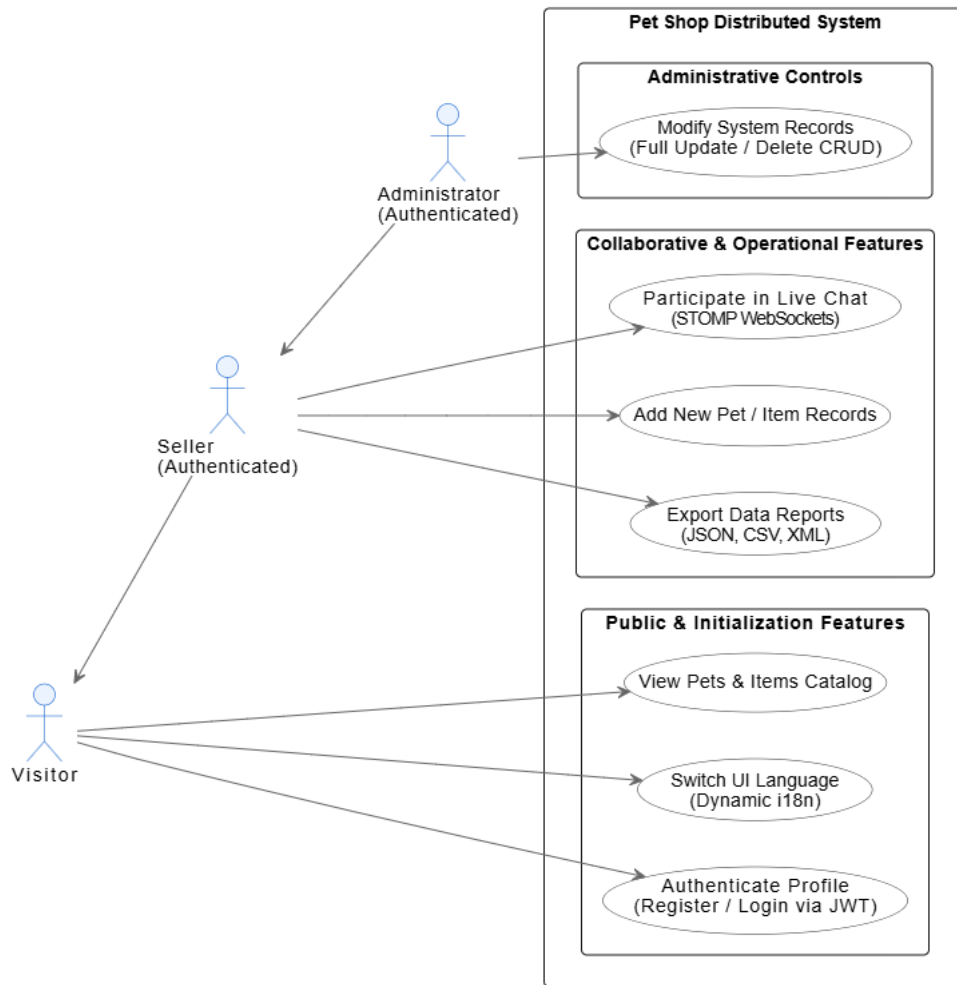


Figure 1: Use-case diagram representing real-time communication and internationalization scopes

3.2 Entity-relationship diagram

The database layer isolates data schemas across backend service instances. Real-time chat messages are managed as temporary, in-memory message packets within the application broker context to maximize messaging performance.

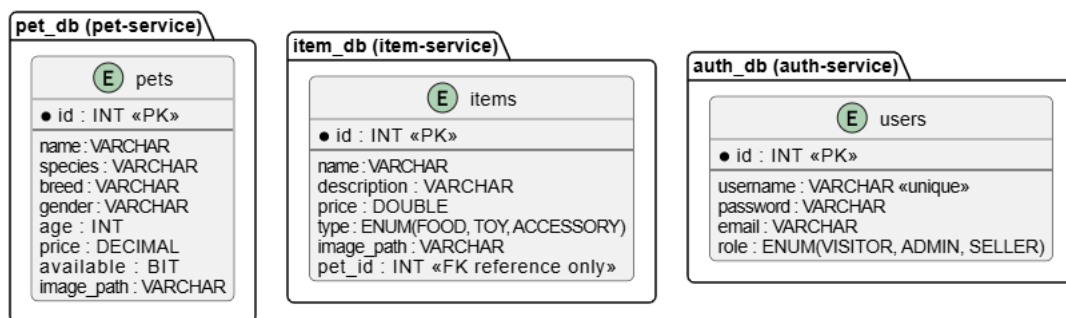


Figure 2: Entity-relationship diagrams

3.3 Class diagram

The core structure includes message models, message broker configs, session event tracking, and runtime language selection interceptors.

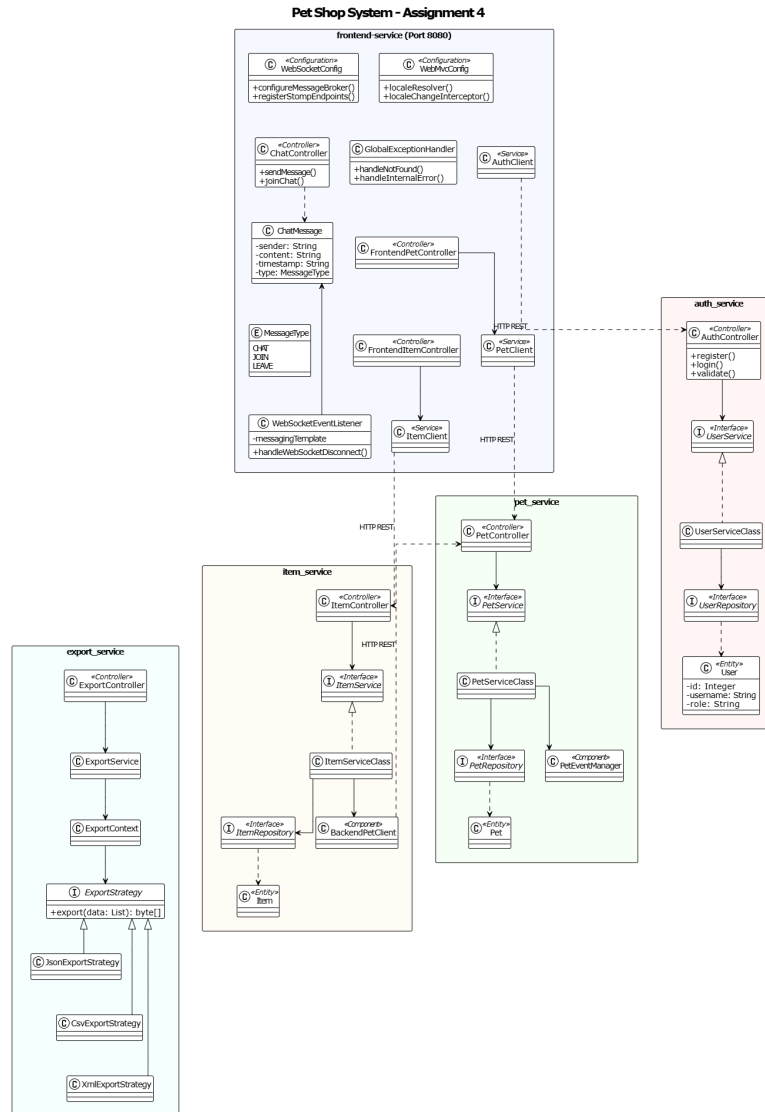


Figure 3: Class diagram showing WebSocket message brokers and internationalization structures

3.4 Sequence diagram

This sequence diagram outlines the end-to-end routing of real-time chat messages from a sending user to the message broker, which broadcasts the payload to all connected clients without requiring a browser page reload.

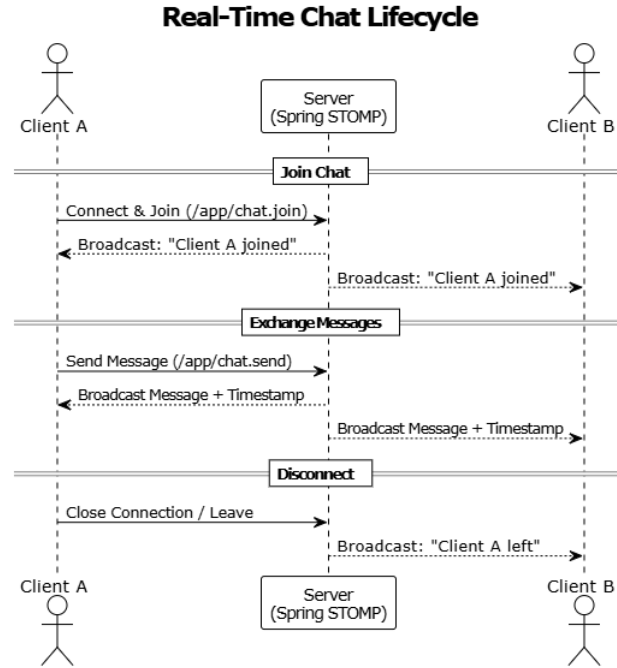


Figure 4: Sequence diagram for WebSocket communication loops and message broadcasting

4 Functional requirements

4.1 Preservation of Core Functionalities (Assignments 1–3)

The application retains all original domain capabilities across the rewritten architecture. Users can search and filter pet records, view item dependencies, register secure profile records, and request system exports in multiple file formats.

4.2 Client-Server Separation API Endpoint Interaction

The application completely decouples client-facing presentations from server business logic. The ‘frontend-service’ web layout renders interface layouts independently, communicating with downstream services via backend REST endpoints. This design isolates data access logic from view rendering components and allows the frontend layer to be deployed as an autonomous application block.

4.3 Dynamic Internationalization (i18n)

Multi-language operations are managed via language bundles configured in the presentation layer, supporting English (‘messages.properties’) and Romanian (‘messages_ro.properties’).

- **Dynamic Locale Filtering** – A `LocaleChangeInterceptor` component intercepts explicit user changes via the navbar elements (e.g., `?lang=ro` or `?lang=en`), storing the preference within a session-scoped cookie context.

- **Template Text Interpolation** – Text headers, table titles, form labels, and operational buttons are resolved dynamically at runtime by parsing resource identifiers using template tag selectors like `th:text="#{nav.home}"`.
- **Dynamic Message Parameterization** – Complex localized patterns containing variable session attributes (such as active user indicators) are constructed securely by passing runtime arguments straight into message tokens:

`Rendered_String = Resource_Key(param1, param2)`

4.4 Real-Time WebSocket Chat Messaging System

The application provides an instant-messaging subsystem that enables real-time user communication without browser refreshes or manual page polling:

- **Transport Protocol Configuration** – The `WebSocketConfig` class maps the persistent connection string boundary to the `/ws-chat` endpoint, allowing standard WebSocket connections or SockJS emulations to initialize safely.
- **Message Mapping and Delivery** – The application routes message frames to dedicated controller entry points based on destination path match strings:
 - `/app/chat.join` – Registers an inbound participant name within the underlying connection attributes map and broadcasts a system join event.
 - `/app/chat.send` – Appends a server timestamp to incoming chat payloads and broadcasts the message packet to the application subscriber array.
- **Lifecycle Event Monitoring** – The `WebSocketEventListener` leverages system alerts to handle user disconnections. If a socket connection closes unexpectedly, the server wraps the username context, builds a structural system message, and notifies all remaining clients over the `/topic/public` channel.
- **UI Thread Rendering** – The client-side script parses incoming text frames and dynamically appends them to the chat DOM tree, showing the sender's identity, timestamp, and message text.

4.5 Distributed Graceful Error Handling and Consistent Payloads

The application implements a global exception handling network to catch system anomalies and prevent unhandled application failure screens:

- **Consistent Error Contracts** – Microservice application processes use uniform error interceptors to standardize error responses. When a resource lookup or input validation check fails, the application returns a consistent data structure:

`Error_Payload = {message, errorCode, timestamp}`

- **Client Presentation Isolation** – The interface catches failing server responses and renders them cleanly within structured error containers. This prevents application page crashes and presents users with helpful feedback text, error badge metadata, and backward navigation fallbacks.

5 Design patterns

5.1 Broker Pattern (Architectural)

The real-time messaging layer relies on the **Broker Pattern**. The application sets up a message broker topology that handles message distribution behind the scenes. Clients don't talk directly to other active client threads; instead, they send message packets to designated routing targets. The STOMP broker handles message routing and safely broadcasts them to all active channel subscribers.

5.2 Mediator Pattern

The **Mediator Pattern** is used to orchestrate interactions within the 'frontend-service' web layout. Instead of allowing client JavaScript functions to call core business microservices directly, components send form data to 'ItemController' and 'PetController' endpoints. These controllers act as mediators, handling authentication checks, assembling fallback data structures if downstream requests fail, and preparing unified models before rendering view components.

5.3 Observer Pattern (Event-Driven)

The **Observer Pattern** is utilized by the application framework to track WebSocket connection states. Components like 'WebSocketEventListener' monitor system context events (such as 'SessionDisconnectEvent'). When these lifecycle events fire, the listener handles cleanup tasks and broadcasts system status alerts without changing the core connection pool configuration.

6 Implementation details

6.1 STOMP Over SockJS Frame Handshaking Loop

The front-end connection logic hooks into the server-rendered DOM template via a JavaScript engine that initiates the transport handshake loop on mount:

```
var socket = new SockJS('/ws-chat');
stompClient = Stomp.over(socket);
stompClient.connect({}, onConnected, onError);
```

Upon validation of the socket handshake, the script subscribes to the public topic destination target string at '/topic/public'. Inbound message frames are intercepted asynchronously, and their contents are rendered directly into the interface using localized CSS alignment styles.

6.2 Global Request Interception and Exception Management

Error redirection is handled dynamically via specialized framework layout handlers. When a connection timeout or page entity resolution check fails across the cluster, the system intercepts the exception and maps it to a tailored template response:

```

@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ModelAndView handleNotFound(ResourceNotFoundException ex) {
        ModelAndView mav = new ModelAndView("error");
        mav.addObject("statusCode", "404");
        mav.addObject("errorCode", "NOT_FOUND");
        mav.addObject("errorTitle", "Page Not Found");
        return mav;
    }
}

```

This architectural safeguard maps backend service failures onto a uniform template view, displaying structured error labels alongside intuitive navigation fallback links.

7 Unit testing

Every application module includes unit test components managed through JUnit 5 and Mockito, testing transactional data logic independent of active database setups:

- **PetServiceTest** – Validates data storage, updates, and sorting algorithms.
- **ItemServiceTest** – Verifies inventory rules and cross-service ID parsing.
- **AuthServiceTest** – Tests user password encryption and token validation workflows.
- **ExportServiceTest** – Asserts data parsing compliance across JSON, CSV, and XML generation classes.
- **NotificationServiceTest** – Confirms async background log processing and mail execution steps.